Article Title

Saksham Diwan[1], Harshaan Chugh[1], Shuo Cao[2], Erik Garrison[2*], Giulia Guidi[1,3*]

[1*]Department of Computer Science, Cornell University, 107 Hoy Rd, Ithaca, 14853, New York, USA.
[2*]Department of Genetics, Genomics and Informatics, University of Tennessee Health Science Center, Memphis, 38163, Tennessee, USA.
[3]Applied Mathematics and Computational Research Division, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, 94720, California, USA.

*Corresponding author(s). E-mail(s): egarris5@uthsc.edu; gg434@cornell.edu;
Contributing authors: sd966@cornell.edu; hsc53@cornell.edu;

Abstract

Motivation: Motivation text.
Results: Results text.
Availability and Implementation: Full URL of the software and data, license, and implementation details.
Contact: egarris5@uthsc.edu, gg434@cornell.edu
Supplementary Information: Supplementary data are available at *Bioinformatics* online.

1 Introduction

Introduction text [1].

2 System and methods

2.1 FastGA

FastGA aligns genome assemblies on a single shared-memory node. It first converts the input FASTA into a compact sequence database (GDB) and builds a genome index (GIX): the sorted list of syncmer-selected 40-mers together with the position, contig and strand of every occurrence. Seeds are found by merging the index with itself, keeping matches whose k-mers occur at most a fixed number of times (10 by default). Seeds are then grouped by contig pair and diagonal, extended into local alignments, and written in FastGA's `.1aln` format, which `ALNtoPAF` converts to PAF.

2.2 DistributedPanGA

DistributedPanGA computes the same self-alignment as FastGA with P MPI ranks. All ranks open the same GDB on a shared file system and read sequence on demand, so no sequence data is broadcast. The pipeline has six stages.

1. **Extraction.** Each rank scans a contiguous range of whole contigs and emits one record per selected k-mer occurrence.
2. **Distributed sort.** Records are exchanged by k-mer prefix and sorted locally, so that the concatenation of all ranks' slices is FastGA's sorted index.

3. **Seed generation.** Each rank runs FastGA's seed merge on its own slice.
4. **Seed redistribution.** Every contig pair is owned by exactly one rank, and each seed is sent to the owner of its pair.
5. **Alignment.** Each rank sorts its seeds and aligns its own contig pairs with FastGA's code.
6. **Output.** Each rank converts its alignments to PAF; the union of the per-rank files is the result.

Selection, seed merging and alignment are FastGA code, kept close to the original so that it can be compared with upstream. The distributed parts are new: the extraction partition, the distributed sort, an adapter that presents a rank's slice through FastGA's index interface, the deterministic assignment of contig pairs to ranks, and the two all-to-all exchanges (Section 3.2).

Our correctness criterion is exact agreement with an independent single-node FastGA run on the same GDB. Both PAF files are sorted lexicographically, keeping duplicate lines, and compared byte for byte. This multiset equality is stronger than matching alignment counts.

3 Algorithm

3.1 Seed extraction

Extraction reproduces FastGA's seed selection exactly. Each rank streams its contigs in windows of up to 10 Mbp and slides over every 12-mer, comparing the canonical 8-mers it contains. A position is selected when the smallest 8-mer lies at either end of the 12-mer (a closed syncmer); ties are kept, as in FastGA.

Each selected position yields two records: the forward 40-mer starting at the position and the reverse-complement 40-mer that ends 12 bases after it. A record stores the 2-bit packed k-mer together with its contig, position and strand. Per-occurrence records are required because seeds are pairs of positions; k-mer counts alone cannot produce alignments.

The 40 bases of a k-mer may cross the boundary between two scan windows. Every window after the first therefore also holds the 40 bases that precede it, which covers all reverse-complement k-mers and all forward k-mers that start before the window. Only a forward k-mer that runs past the right edge of a non-final window is read separately from the GDB. The selection loop, the selected positions and the order in which records are emitted are identical for every window size.

3.2 Data movement between ranks

The pipeline moves data between ranks twice. In the distributed sort, each record is assigned to one of 1,024 buckets by the first five bases of its k-mer. A global histogram, obtained by a single reduction, gives every rank the same bucket counts, and each rank takes a contiguous range of buckets holding about $1/P$ of the records. After the exchange, each rank sorts its records and recomputes the longest-common-prefix values at the boundaries with its neighbours, so the slices together form the global sorted index.

In seed redistribution, each seed is sent to the rank that owns its contig pair. Every rank computes the same pair-to-rank map independently from the contig lengths, by assigning pairs in decreasing order of length product to the least-loaded rank. No communication is needed to agree on it.

Both exchanges are personalized all-to-all transfers of fixed-size records, which can exceed the 32-bit counts of `MPI_Alltoallv` (about 2 GiB per rank). We therefore track sizes in 64 bits and, when a transfer exceeds a per-round budget, split it into rounds. Each round sends at most a fixed number of whole records to each peer, and every slice is written at its final offset in the receive buffer. The received bytes are therefore identical to those of a single unbounded exchange, independent of the number of rounds, while staging memory is bounded by the budget. Seeds are held in memory throughout, so the send and receive seed buffers coexist during redistribution (Section 4).

4 Implementation

4.1 Extraction I/O

Per-rank timers showed that extraction dominated early runs and scaled poorly with the number of ranks. System-call tracing located the cause in the packing step. The scan already held the current window in memory, but the 40 bases of each selected k-mer were read again from the GDB, which costs one file seek per record.

We now pack each k-mer directly from the window buffer, reading from the GDB only in the boundary case of Section 3.1; the window refill was extended by 40 bases of left context. Both paths use the same packing routine, and the selection loop is unchanged, so the change affects only where bases are read from.

We validated the change at three levels. The extraction output is byte-identical to the previous implementation and to a reference dump from an instrumented copy of FastGA's index builder. Unit tests run with a 64-base window to force many boundary cases, check that both packing paths are exercised and together account for every record, and also run under address and undefined-behaviour sanitizers. Finally, end-to-end runs match the FastGA reference alignments.

5 Discussion

5.1 Limitations and future work

Alignment is now the dominant stage, and its cost is unevenly distributed across ranks. The time of an alignment task follows the amount of sequence it aligns, which is not known in advance and is poorly predicted by seed counts or contig lengths. This suggests replacing the static assignment of contig pairs with dynamic scheduling, in which ranks take the next task from a shared queue as they become free.

Scheduling alone has a limit: each contig pair is aligned by one rank, and self-alignments of long contigs are the largest tasks. Scaling further requires splitting such a task across ranks while producing exactly FastGA's alignments. This is an open problem.

Two implementation limits also bound scale. The pair-to-rank map uses 8-bit entries, which caps the number of ranks at 127, and extraction partitions work at whole-contig boundaries.

Finally, exact validation at human scale requires an independent single-node FastGA reference, whose cost grows rapidly with the number of haplotypes. Agreement between runs at different rank counts is weaker evidence than agreement with FastGA, because it shares any error of our own pipeline.

Acknowledgements.

Author contributions.

Supplementary data. Supplementary data are available at *Bioinformatics* online.

Conflict of interest. None declared.

Funding. This work was supported by the Full Agency Name [grant number xxxx].

Data availability.

References

[1] Author, F. & Author, S. Example article title. *Journal Name* **1**, 1–10 (2026).